TrustVault: Database Schema and Deployment Guide

SETUP | Snovasys Fresher Hiring Challenge 2026

1. Technologies

Part	Technology
Frontend	React and Vite
Backend	Node.js and Express
Database	MySQL
Login	JSON Web Tokens (JWT)
Password hashing	bcrypt / bcryptjs
Testing	Jest and Supertest
Version control	Git and GitHub
Hosting	Render for the web application, Aiven for the managed MySQL database

2. Live application and demo accounts

The deployed application is at https://trustvault-lh94.onrender.com. The health check is at https://trustvault-lh94.onrender.com/api/health. It should answer with the status of the application and of the database connection.

If the service has been idle, the first request can be slow on some hosting plans, so please wait a little and refresh once.

These are the demo accounts for the live site:

Account	Email	Password
Admin	23jr1a05h4@gmail.com	snovasys@123
User 1	bhanu@gmail.com	bhanu@123
User 2	prakash@gmail.com	prakash@123

The admin account can open the audit log and the admin view of document activity. User 1 and User 2 are normal accounts. Any logged-in user can upload a document and becomes its owner, and the owner can give another user read or edit access. A simple way to see the permissions is to upload a file with User 1, share it with User 2, and then remove the access again.

These passwords are only for this demo. I do not use them anywhere else.

3. Database overview

TrustVault uses a relational database for the user accounts, the document details, the permissions and the audit logs. There are four main tables.

users

Stores the accounts. Main fields are the id, the name, a unique email, the password hash, the role and the time the account was created. Only password hashes are stored. The email must be unique. The hash is never returned in an API response.

documents

Stores the details of each document. Main fields are the id, the owner id, the file name, the storage key (the name used to find the stored file), the content type and the upload time. Each document always has an owner. The access check is done before the file is returned, and a document ID alone is never treated as permission. The database row and the stored file must stay consistent.

document_permissions

Stores who can use a document in addition to its owner. Main fields are the id, the document id, the user id, the permission level (read or edit) and the time it was created. The references are checked by foreign keys, the permission is checked on every protected request, and only a person who is allowed to manage the document can give or remove it.

audit_logs

Stores the important actions. Main fields are the id, the user, the document, the action, the details and the time. Access to it is limited, and passwords, secrets and document contents are never written into it. Denied attempts are recorded as well, where the audit code covers them.

The field names above describe the data model in general. The real names and types are in the SQL schema file of the repository, and that file is the one to use.

4. Setting up the database

You need a MySQL server (local or managed), the database credentials, and Node.js with npm.

Step 1. Create the database. On a local MySQL it can be done like this:

```
CREATE DATABASE trustvault;
USE trustvault;
```

Step 2. Create the tables by running the SQL schema or the setup scripts from the repository. Please use the schema file from the repository and do not write the tables again

by hand.

Step 3. Create the file server/.env and fill in the variables that server/db.js and the login code expect:

```
DB_HOST=
DB_PORT=
DB_USER=
DB_PASSWORD=
DB_NAME=trustvault
DB_SSL=
JWT_SECRET=
PORT=
```

For the deployed application, use the connection details that the database provider gives. If SSL is needed, set it up the way the existing connection code expects. The .env file must never be committed to GitHub.

5. Running from a clean clone

1. Clone the repository: git clone <repository-url>
2. Go into the folder: cd TrustVault
3. Install the frontend packages: npm --prefix client install
4. Install the backend packages: npm --prefix server install
5. Create server/.env with the database and login settings.
6. Run the SQL setup script from the repository to create the tables.
7. Start the backend: cd server, then npm start. If package.json uses another script name, use that one.
8. In a second PowerShell window start the frontend: cd client, then npm run dev.
9. Open the address that Vite prints, usually http://localhost:5173, and check that the frontend can talk to the backend and that the database connects.

6. Deployment

The web application is deployed on Render and the backend connects to a managed MySQL database on Aiven. I deploy in this order:

1. Push the changes to GitHub.
2. Check the build and start commands in Render.
3. Add the environment variables in the Render settings (not in the code).
4. Start a deploy and read the deployment logs for errors.
5. Open the health endpoint and check that the application and the database are reported correctly.

6. Log in and try the protected document actions.
7. Test authorization and the audit log, and then upload and download a file.

I should open the live link in a private window and on another device before submitting. If the link does not open for the judges, those marks are lost.

7. Where the files are stored

The database keeps only the details of a document. The real file is stored separately. If the files are on a temporary disk of the hosting service, they can disappear after a restart or a new deployment, while the database rows stay. A working database connection therefore does not prove that a file exists.

For real use, the files should be placed in storage that is persistent, private and not reachable around the Express permission checks. I have not solved this fully, and I say so in THREATS.md.

8. Secrets

- Keep server/.env out of Git.
- Use environment variables for the database credentials and the JWT secret.
- Do not put real production passwords in the code.
- Do not publish private database certificates or credentials.
- Use strong secrets, and change any secret that may have been exposed.
- Set production secrets in the settings of the hosting provider.

9. Checklist before submitting

Check	Done
Repository clones without errors	
Frontend packages install	
Backend packages install	
Environment variables are written in the README	
Database setup scripts are in the repository	
Tables can be created from the scripts	
Backend starts	
Frontend starts	
Health endpoint answers correctly	
Login works	
Another user's document is refused	
Sharing with permission works	
Audit logs are created	
Upload and download are tested	
No secrets are committed	

10. Common problems

Problem	What to check
Backend cannot connect to the database	Host, port, user, password, database name and the SSL setting. Also check that the database service is running.
Site opens but login fails	JWT_SECRET must be set in Render. Check the network tab for the real error.
Upload works but download fails	The stored file may be missing because of the temporary disk. Check the storage section above.
Frontend cannot reach the backend	Check the API address in the frontend and the allowed origin in the backend.

11. Conclusion

This guide describes how to set up and deploy TrustVault. A really clean clone only works if the repository contains the actual schema or setup scripts and all setup steps that are not secret, so I make sure that they are there before I submit.